PROJET PÉDAGOGIQUE · SECURE BOOT ET CRA

Prouver ce que fait un appareil connecté

Chaîne de confiance embarquée, mises à jour sûres et conformité CRA, démontrées en Python.

HAB · FIT signé · dm-verity · Mender · CRA

Mike · Master cybersécurité

LE PROBLÈME

Un appareil déployé peut être trafiqué

Le risque

Un appareil chez le client.

Un attaquant remplace son logiciel, par accès physique ou par une fausse mise à jour.

L'obligation

Le CRA (règlement UE).

Le fabricant doit protéger le produit et corriger vite une faille. Dates à confirmer sur EUR-Lex.

La question

Comment être sûr ?

Que l'appareil n'exécute que le code du fabricant, au démarrage comme après chaque mise à jour.

LA SOLUTION

Une chaîne où chaque maillon vérifie le suivant

Puce
Clé gravée dans les fusibles (HAB)

→

U-Boot
Vérifié par la puce

→

Noyau
Vérifié par U-Boot (FIT)

→

Système
Vérifié bloc par bloc (dm-verity)

Et pour les mises à jour : Mender

Installation seulement si la signature est valide, sur une copie B du système. Si ça plante, retour automatique sur A.

Un seul secret matériel au départ ; tout le reste se vérifie par signature.

LES PREUVES

Chaque attaque rencontre une protection précise

Attaque	Ce qui la bloque	Scénario
Remplacer U-Boot	HAB : hash de clé dans les fusibles	2 et 3
Modifier le noyau	FIT : hash des images	4
Recombiner des images valides	FIT : signature de la configuration	5
Modifier le système de fichiers	dm-verity : root hash signé	6
Fausse mise à jour	Mender : signature de l'artefact	8
Mise à jour qui plante	Mender : retour automatique sur A	9
Rejouer une vieille version	Anti-rollback (option)	10

Un système neuf et valide passe toute la chaîne : c'est le scénario 7.

LA DÉMO

Les attaques rejouées en dix secondes

Démo Python

Un seul fichier, 10 secondes.

- 11 scénarios, 11 conformes.
- Vraies signatures RSA-2048.
- Windows, Linux, sans carte.

À montrer en direct

- **2** : même attaque, refusée en closed, acceptée en open.
- **5** : le contrôle naïf passe, la signature de configuration refuse.
- **9** : image signée qui plante, retour sur A.

Lancer

```
powershell -ExecutionPolicy
Bypass `
-File .\run_demo.ps1 -Pause
python demo_chain.py --
only 5
```

Modèle pédagogique : mêmes décisions que le matériel, exécutables sur un laptop.

LAB RÉEL ET CONFORMITÉ

Ce qui a vraiment tourné, et ce que ça couvre du CRA

6029
tâches Yocto
Image Linux avec Mender.

273
composants
SBOM CycloneDX.

A/B
démarrage
QEMU, artefact signé validé.

Lien avec le CRA

- **Sécurité dès la conception** : HAB, FIT, dm-verity.
- **Mises à jour de sécurité** : Mender signé, retour arrière.
- **Vulnérabilités** : SBOM, puis alerte sous 24 h et notification sous 72 h.

Limite assumée : l'image QEMU n'a ni HAB ni dm-verity ; ces couches sont montrées par la démo Python. Délais CRA à re-vérifier sur EUR-Lex.

POUR CONCLURE

Limites et plan pour la Nitrogen8M

Limites

- Démo = modèle : pas de vrais fusibles.
- HAB réel : outils NXP (CST), gravure irréversible.
- Dates et qualification CRA à confirmer.

Pendant le stage

- Générer les clés, signer SPL et U-Boot.
- Vérifier en carte open (hab_status).
- Intégrer FIT, dm-verity et Mender dans Yocto.
- Fermer la carte en dernier.

Merci. Questions ?